

Logan Mooneyham

Professor Kanemoto

CPSC-39

4/24/26

### Final Project Report

My final project is a Java application called the Study Flashcard App. The purpose of this program is to help users create, organize, and study digital flashcards in a structured way. It is functionally similar to flashcard apps like Quizlet or Brainscape

Many students use paper flashcards for memorization, but paper cards, as with any physical medium for study, can be lost, damaged, or difficult to organize in a way that does not require a hassle like many rubber bands or having to stuff them into a backpack pocket that they don't quite fit. This program provides a digital alternative that allows users to store flashcards, review them repeatedly, and keep their study materials saved between sessions. The application can be used for vocabulary words, formulas, dates in history, science facts, or any topic that requires repetition and recall. It can even be used for activities outside of studying such as fun trivia games. The questions and answers can be anything the user wants them to be.

The program is operated through a console-based menu system. When the user starts the application, they are given options to add flashcards, view all saved flashcards, remove

flashcards, begin a quiz, view quiz statistics, save flashcards to a file, load flashcards from a file, or exit the program. This eight-optioned design keeps the program simple and easy for users to interface with while still providing useful functionality. The program's key practicality is its ability to help users study in an organized and repeatable way.

To keep the project organized, I used object-oriented programming and separated responsibilities into multiple Java classes, as recommended by our professor.

- The Main class controls the overall program flow and displays the menu.
- The Flashcard class represents one flashcard and stores a question, answer, and topic.
- The FlashcardManager class stores and manages all flashcards using an ArrayList.
- The QuizEngine class handles quiz mode by selecting random cards and checking answers.
- The Statistics class tracks the number of correct and incorrect answers and calculates quiz performance.
- The FileHandler class is responsible for saving and loading flashcards from a file.

Separating the program into classes made the code easier to read, debug, and expand. This is a widely accepted principle of coding anyway. One giant class with every facet of the code contained within it would be sloppy, hard to parse through, and impractical.

Several data structures were used in this project, the most important data structure being the ArrayList, which stores all flashcard objects in the FlashcardManager class. An ArrayList was

chosen because it can grow dynamically as users add more flashcards, unlike a fixed-size array. This choice also makes it easy to access flashcards by index, remove items, and loop through the collection. The program also makes extensive use of Strings, since questions, answers, topics, and file data are all text-based. Another structure used is a simple text file that stores flashcards permanently between sessions, each flashcard being saved as one line using a delimiter format so the data can be reconstructed later.

The first major algorithm in the program is the random flashcard selection algorithm used in quiz mode. The program first determines the number of flashcards in the ArrayList and then generates a random index between zero and the last position in the list, returning the flashcard stored at that index. This prevents users from memorizing card order and improves the study experience by introducing randomness, and because ArrayList allows direct index access, the time complexity of this operation is  $O(1)$ .

The second algorithm is the display-all-flashcards algorithm, which is used when the user chooses to review all cards. The program iterates through the ArrayList from start to finish, printing each flashcard's topic, question, and answer. Since each element must be visited once, this is a linear traversal with a time complexity of  $O(n)$ , where  $n$  is the number of flashcards.

The third algorithm is the save-to-file process, which runs when the user saves or exits the program. The program opens a text file and loops through each flashcard, writing it in the format "Question | Answer | Topic," before closing the file. A related loading process reads each line, splits the data, creates Flashcard objects, and adds them back into the ArrayList. Both operations run in  $O(n)$  time since every flashcard must be processed.

Overall, performance depends on the operation being performed. Adding a flashcard is  $O(1)$ , while viewing, saving, loading, and deleting flashcards are  $O(n)$  due to full traversal or shifting elements. Random selection remains the most efficient operation at  $O(1)$ .

Several issues were addressed during development. The most prominent was the need for the addition of input validation using `hasNextInt()` to prevent crashes from invalid menu input. File persistence was implemented through the `FileHandler` class to ensure flashcards are saved and reloaded between sessions.

The project used Java, Eclipse IDE, GitHub Classroom, and standard Java libraries such as `ArrayList`, `Scanner`, `FileReader`, `FileWriter`, `BufferedReader`, and `PrintWriter`. These tools were chosen (partially per the recommendations of both Grok and ChatGPT) for their reliability and suitability for a console-based application.

Future improvements could include creating a GUI for a graphical user interface, or options for flashcard editing, topic-based search, multiple user support, and advanced quiz modes such as timed or difficulty-based quizzes.

In conclusion, the Study Flashcard App demonstrates effective use of object-oriented programming, data structures, file handling, and algorithm design, resulting in a functional study tool that also reflects strong software development practices.